Breadth-First Search and Depth-First Search on a Search Tree

Name: D. Jeshurun Silas

Registration No: 192525324

Activity – 9 : Demonstrating BFS and DFS on the given state space tree

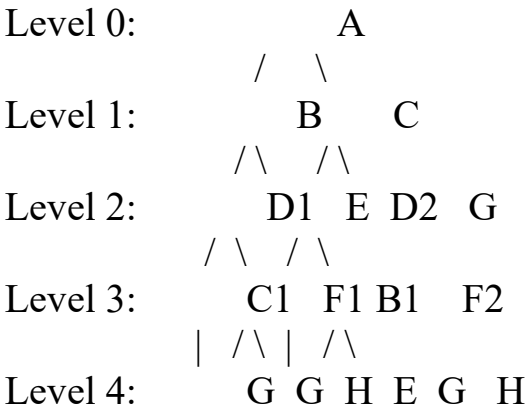
---

1. Introduction

Uninformed (blind) search strategies explore a state space without any knowledge of how close a state is to the goal. Two of the most basic ones are BFS and DFS. Both start at the root node A and expand nodes, but they differ in the order in which they do so.

2. The Given Search Tree

To tell repeated labels apart, repeated nodes are numbered by position (the tree has 17 nodes in total).



- Children of **A**: B, C
- Children of **B**: D1, E; children of **C**: D2, G
- Children of **D1**: C1, F1; children of **D2**: B1, F2
- **C1** → G; **F1** → G, H; **B1** → E; **F2** → G, H

Children are always taken from left to right.

3. Breadth-First Search (BFS)

Idea: Explore the tree level by level. All nodes at depth d are expanded before any node at depth d+1.

Data structure: A FIFO queue. New children are added at the back, and the next node is taken from the front.

Step-by-step trace

Step	Node expanded	Queue after expansion (front → back)
1	A	B, C
2	B	C, D1, E
3	C	D1, E, D2, G
4	D1	E, D2, G, C1, F1
5	E	D2, G, C1, F1 (leaf)
6	D2	G, C1, F1, B1, F2
7	G	C1, F1, B1, F2 (leaf)
8	C1	F1, B1, F2, G
9	F1	B1, F2, G, G, H
10	B1	F2, G, G, H, E
11	F2	G, G, H, E, G, H
12–17	G, G, H, E, G, H	All leaves; queue empties

BFS visiting order:

A → B → C → D → E → D → G → C → F → B → F → G → G → H → E → G → H

4. Depth-First Search (DFS)

Idea: Go as deep as possible along one branch. When a leaf is reached, backtrack to the nearest node that still has unexplored children.

Data structure: A LIFO stack (or recursion). Children are pushed right to left so the leftmost child is popped first.

Step-by-step trace

Step	Node expanded	Stack after expansion (top on left)
1	A	B, C
2	B	D1, E, C
3	D1	C1, F1, E, C
4	C1	G, F1, E, C
5	G	F1, E, C (leaf, backtrack)
6	F1	G, H, E, C

Step	Node expanded	Stack after expansion (top on left)
7	G	H, E, C (leaf)
8	H	E, C (leaf)
9	E	C (leaf; left subtree of A done)
10	C	D2, G
11	D2	B1, F2, G
12	B1	E, F2, G
13	E	F2, G (leaf)
14	F2	G, H, G
15	G	H, G (leaf)
16	H	G (leaf)
17	G	Empty (leaf)

DFS visiting order:

A → B → D → C → G → F → G → H → E → C → D → B → E → F → G → H → G

5. Side-by-Side Comparison

Feature	BFS	DFS
Traversal pattern	Level by level	Branch by branch (deep first)
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Memory	High: stores a whole level (up to b^d nodes)	Low: stores only one path (b·m nodes)
Completeness	Yes (finite branching)	No on infinite or cyclic spaces; yes on finite trees
Optimality	Yes (shallowest goal; optimal with uniform step cost)	No
Time complexity	O(b^d)	O(b^m)

Here b is the branching factor (2 in this tree), d is the depth of the shallowest goal, and m is the maximum depth (4 in this tree).

6. Illustration with a Goal Node

Suppose the goal is H.

- **BFS** finds H at depth 4. It is the 14th node visited, after exploring all of levels 0–3 and part of level 4.
- **DFS** finds H at depth 4, but it is the 8th node visited, because the leftmost deep branch happens to lead there quickly.

DFS was faster in this case only because of where the goal sits. If the goal were on the right side of the tree, DFS would have to exhaust the whole left subtree first. BFS is guaranteed to find the shallowest goal, while DFS depends on tree layout.

7. Conclusion

BFS and DFS both visit every node of this tree, but in different orders: BFS gives A, B, C, D, E, D, G, … and DFS gives A, B, D, C, G, F, …. BFS guarantees the shallowest solution but needs more memory, while DFS is memory-efficient but may go down long, unproductive paths. The choice depends on the problem: BFS when the shortest path matters, DFS when memory is limited or solutions are deep.